Project Report

Personal Finance Advisor Bot

A Full-Stack AI-Powered Personal Finance and Budget Management System with Localised INR Accounting

Student Name :Ritesh Patode , Branch : Artificial Intelligence and Data Science (AIDS)

Project Domain AI Specialist – BFSI (Banking, Financial Services and Insurance)

Technology Python, Flask, SQLite, SQLAlchemy, Google Gemini API

Source Code :

https://github.com/PatodeRitexh/ai-based-finance-bot.git

Item	Description
Project domain	AI Specialist – BFSI (Banking, Financial Services and Insurance)
Backend	Python 3.11+, Flask, Flask-SQLAlchemy, SQLite3, Flask-CORS, Werkzeug
Frontend	HTML5, CSS3, Vanilla JavaScript (ES6+), Chart.js
AI integration	Google Gemini API through the official google-genai SDK, with a deterministic rule-based fallback
Architecture	Modular Flask blueprints in a three-tier layered design
Currency	Indian Rupee (INR, ₹) with Indian digit grouping, for example ₹1,50,000.00

1. Abstract

The Personal Finance Advisor Bot is a full-stack web application that records a user's income and categorised expenses, compares spending with monthly budgets, tracks savings goals and produces written financial advice through the Google Gemini API. All monetary values are stored and displayed in Indian Rupees (INR, ₹) using the Indian digit-grouping convention.

The server is written in Python 3.11 with Flask and is organised as a set of blueprints. Data is persisted in SQLite through Flask-SQLAlchemy in five entities: users, incomes, expenses, budgets and savings_goals. Every query on user data is restricted to the authenticated user. Expenses are classified into nine fixed categories. A service layer computes total income, total expenses, net savings, savings rate, budget utilisation and a six-month cash-flow series, and the browser presents these figures with Chart.js doughnut and bar charts.

The advisory component sends a structured prompt containing the user's figures for the selected month to a Gemini model at a temperature of 0.2 and requires a reply in a fixed JSON schema. Transient failures, such as HTTP 503 responses, are retried with exponential backoff. If the retries fail or the reply cannot be parsed, a deterministic rule engine based on the 50/30/20 allocation rule produces the analysis, so the application keeps working when the external service is unavailable.

The system was verified with an automated suite of eleven test cases (test_app.py), all of which passed, and with a demonstration data set for a user with a monthly income of ₹1,08,000.00, expenses of ₹64,500.00 and net savings of ₹43,500.00, which is a savings rate of 40.28%. The output of the application is informational and does not constitute professional financial advice.

Keywords: personal finance, budget management, expense tracking, savings goals, Flask, SQLAlchemy, Google Gemini API, Chart.js, Indian Rupee, BFSI.

2. Introduction

2.1 Background of Personal Finance and BFSI Technology

Personal finance management is the practice of planning, recording and controlling an individual's income, spending, savings and financial goals. Its basic activities are recording income, classifying expenses, comparing actual spending with a plan, and setting aside part of the income for emergencies and for planned purchases. Regular practice of these activities allows a household to meet recurring obligations such as rent and utilities, to absorb unexpected costs, and to reach larger targets, such as a professional certification or a travel plan, without borrowing.

In the Banking, Financial Services and Insurance (BFSI) sector, retail customers now interact with money largely through software. Account statements, card transactions and Unified Payments Interface (UPI) payments produce a continuous stream of small debits across many categories. The ease of making a payment has not been matched by an equal ease of understanding the overall position. A person may know the balance of an account and still not know what share of the month's income went to housing, food or discretionary purchases.

Financial institutions and technology companies therefore treat budgeting, categorisation and goal tracking as standard features of retail banking applications. This project addresses the same need in a

self-contained application that can be run and inspected by a student, and it applies the Indian conventions of currency symbol, digit grouping and monthly salary cycles throughout.

2.2 Limitations of Traditional Trackers and the Need for AI Advisory

A conventional expense tracker, whether a notebook, a spreadsheet or a simple mobile application, stores numbers and leaves their interpretation to the user. This produces four recurring limitations.

1. **No interpretation.** A tracker shows that ₹3,850 was spent on entertainment but does not state whether this is high for the user's income, or whether it breaches a limit the user intended to keep.
2. **No early warning.** A budget that is checked only at the end of the month reveals an overspend after it has happened. A threshold alert issued while the month is in progress allows the user to adjust spending.
3. **Disconnected goals.** Savings goals are often recorded separately from the ledger, so progress towards a goal is not linked to the money that is actually saved each month.
4. **Generic advice.** Printed tips and published rules of thumb do not take account of the user's own figures, such as a high rent share or an irregular freelance income.

A large language model can read a structured summary of a month's finances and produce an explanation in plain language, together with specific recommendations. The language model is not a replacement for a licensed adviser, and its output is not guaranteed to be consistent between runs. For this reason the project places the model behind a validation layer and a deterministic rule engine, so that the arithmetic is always computed by the application and the model supplies only the commentary.

2.3 Overview of the Proposed Full-Stack System

The Personal Finance Advisor Bot is a Flask application with its own ledger. A registered user records income entries and expense entries, each with an amount, a date and a description. Expenses belong to one of nine categories. The user sets a budget limit per category and per month, and creates savings goals with a target amount and a target date.

A dashboard aggregates the ledger for a selected month and shows total income, total expenses, net savings, savings rate, category distribution, budget status and a six-month cash-flow history. When the user requests an analysis, the server builds a prompt from the same aggregated figures and sends it to the Google Gemini API. The reply is validated and returned to the browser as structured advice. If the service cannot be reached after retries, the server produces an analysis from a rule engine instead.

The system is built as a modular monolith. Routes, models and services are separated into packages, and each functional area is registered as a Flask blueprint. This keeps the code readable, allows each module to be tested separately, and leaves a clear path to extend the application, as described in Chapter 17.

3. Problem Statement

Individuals find it difficult to manage money consistently because income arrives at intervals, expenses occur in many small amounts across several categories, and there is rarely an up-to-date view of how much remains available to save. Existing tools either record figures without interpreting them or offer generic advice that is unrelated to the user's own ledger.

The problem addressed by this project is to design and implement a web application that (a) maintains an independent ledger of a user's income and categorised expenses, (b) compares the ledger with category budgets and savings goals, and (c) produces personalised advice from the user's own figures, while remaining usable if the external AI service fails. Table 1 lists the principal difficulties and the response of the system to each.

Table 1: Common difficulties in personal finance management and the corresponding system solutions

Difficulty	Effect on the user	System solution
Manual and incomplete record keeping	The monthly position is unclear and records are abandoned	Dedicated ledgers for income and expenses with create, edit and delete operations; totals are calculated by the server
Unclear spending pattern	The user cannot tell which category absorbs most of the income	Category distribution shown as a Chart.js doughnut chart across nine standard categories
Budgets exceeded without notice	Overspending is discovered after the month has ended	Per-category monthly budgets with a Warning status above 80% and an Exceeded status above 100% of the limit
Vague savings goals	Progress cannot be seen and the target date is forgotten	Savings goals with target amount, saved amount, progress percentage, target date and a quick deposit action
Generic advice	Tips do not reflect the user's income, rent share or savings rate	Gemini receives the user's monthly figures in INR and returns structured, figure-specific recommendations
Loss of context across months	The user cannot see whether cash flow is improving	Six-month rolling income, expense and net-savings series shown as a bar chart
Dependence on an external AI service	An outage or quota limit leaves the user without any result	Retry with exponential backoff, followed by a deterministic fallback engine, so that the analysis endpoint does not fail
Unauthorised access to financial data	Another person could read or change the records	Hashed passwords, session-protected routes and user-scoped database queries

4. Objectives

4.1 Primary Objectives

1. **Maintain an independent ledger.** Allow a registered user to create, read, update and delete income entries and expense entries, with validation of amounts, dates and categories.
2. **Classify expenditure.** Assign every expense to one of nine standard categories: Food, Rent, Transport, Education, Healthcare, Entertainment, Shopping, Utilities and Other.

3. **Calculate financial indicators on the server.** Compute total income, total expenses, net savings and savings rate for any selected month, and format all amounts in INR with Indian digit grouping.
4. **Plan and monitor budgets.** Let the user set a monthly limit per category, and classify each budget as on track, warning or exceeded according to fixed thresholds.
5. **Support savings goals.** Record goals with a target amount and target date, show the percentage achieved, and allow a quick deposit that increases the saved amount.
6. **Provide AI advisory.** Use the Google Gemini API to produce a monthly summary, spending insights, overspending categories, budget recommendations, saving suggestions and expense reductions in a fixed JSON schema.
7. **Visualise the data.** Present category distribution and six-month cash flow with interactive Chart.js charts.

4.2 Non-Functional and Security Objectives

1. **Credential protection.** Store passwords only as salted hashes produced by Werkzeug, and never return the hash in any response.
2. **Access control.** Protect every private page and API route with an authentication check, and filter every record query by the identifier of the signed-in user so that one user cannot read or modify another user's data.
3. **Availability of the advisory function.** Complete every analysis request with a valid result, from Gemini where possible and from the rule engine otherwise.
4. **Configuration hygiene.** Keep the secret key and the Gemini API key in environment variables loaded from a .env file that is excluded from version control.
5. **Maintainability.** Separate routes, models and services into packages, register features as blueprints, and cover the main behaviour with an automated test suite.
6. **Usability and consistency.** Provide a restrained, consistent interface with accessible contrast, rectangular controls and a layout that adapts to narrow screens.
7. **Regulatory transparency.** Publish a privacy policy and terms of service, and attach an advisory disclaimer to every AI-generated analysis.

5. Scope of the Project

The scope of the project covers a complete, locally deployable personal finance application with a manual ledger and an AI advisory module. The boundaries are stated below so that the capabilities and the limits of the system are clear to the reader.

5.1 In-Scope Capabilities

- **Account management.** Registration, login and logout with hashed passwords and a server-side session check on every private route.
- **Income ledger.** Create, list, update and delete income entries with amount, source, date and description, filtered by month.
- **Expense tracker.** Create, list, update and delete expense entries in nine categories: Food, Rent, Transport, Education, Healthcare, Entertainment, Shopping, Utilities and Other.

- **Budget planner.** One monthly limit per user and category, stored by upsert, with utilisation percentage and a status of on track, warning (above 80%) or exceeded (above 100%).
- **Savings goals.** Goals with a target amount, a current amount and a target date, progress percentage, and a quick deposit action that adds a contribution to the saved amount.
- **Dashboard and analytics.** Monthly totals, savings rate, category distribution (doughnut chart) and six-month cash flow (bar chart), all computed on the server.
- **AI advisory.** A request to the Google Gemini API with a structured prompt, a validated JSON reply, retry on transient errors and a deterministic fallback engine.
- **Localisation.** Indian Rupee symbol and Indian digit grouping in every amount shown to the user.
- **Compliance pages.** A Privacy Policy and Terms of Service available without login.
- **Demonstration and testing.** A seed script that loads a realistic data set and an automated test suite.

5.2 Out-of-Scope Boundaries

- **Professional advice.** The application gives informational guidance only. It does not give investment, tax, loan or insurance advice, and it makes no claim that its recommendations are accurate or suitable for a particular person.
- **Automated trading and transactions.** The system never moves money, places orders, buys securities or connects to a broker or a payment gateway.
- **Bank integration.** There is no link to bank accounts, UPI applications or account aggregators, and no automatic import of transactions. All entries are made by the user.
- **Multiple currencies.** Only INR is supported.
- **Production deployment.** The project uses the Flask development server and a local SQLite file. Horizontal scaling, a managed database, HTTPS termination and administration features are not part of this version.
- **Custom categories.** The nine categories are fixed in the application.

6. Technology Stack

The project uses open-source technologies that can be installed with pip and run on a student workstation without a separate database server. Table 2 lists each technology with its role and the reason it was chosen. The documentation of each component is given in the References [1]–[10], [16] and [17].

Table 2: Technology stack with role and justification

Technology	Layer	Role in the project	Justification
Python 3.11+	Language	Routes, models, calculations, prompt construction and response handling	Readable syntax, mature standard library and first-class SDK support for the Gemini API
Flask	Web framework	Application factory, blueprints, request handling, JSON responses and template	Small core that keeps the code explainable; blueprints give a modular structure

Technology	Layer	Role in the project	Justification
		rendering	without a heavy framework
Flask-SQLAlchemy	ORM	Maps the User, Income, Expense, Budget and SavingsGoal classes to tables and runs queries	Queries are written as Python and parameterised, which removes hand-built SQL strings
SQLite3	Database	Stores all records in one local file under the instance folder	No server process, file-based backup and sufficient for a single-user or demonstration workload
Flask-CORS	Middleware	Sets cross-origin headers for the API routes	Allows the API to be called from a separate front-end origin during development
Werkzeug	Security	generate_password_hash and check_password_hash for salted password hashing	Maintained with Flask; avoids a custom hashing implementation
python-dotenv	Configuration	Loads SECRET_KEY, the Gemini API key and the model name from a .env file	Keeps secrets out of source code and version control
google-genai SDK	AI	Client for the Gemini generate_content call	Official Google SDK for the Gemini API
Jinja2	Template engine	Renders base.html and the page templates	Included with Flask; template inheritance gives all pages the same layout
HTML5	Frontend	Structure of the landing, login, registration, dashboard, privacy and terms pages	Semantic elements and native form validation
CSS3	Frontend	Custom design system with slate and emerald colours, 6px corners and responsive grid	No framework dependency; full control of the visual rules in Chapter 13
Vanilla JavaScript (ES6+)	Frontend	Fetch calls to the API, form handling, table rendering and chart updates	No build step; one file per feature page
Chart.js	Visualisation	Doughnut chart of category distribution and bar chart of	Canvas-based, documented, and supports tooltips and

Technology	Layer	Role in the project	Justification
		six-month cash flow	responsive resizing

6.1 Notes on Selected Choices

SQLite and SQLAlchemy. The relationships in this application are simple: one user owns many incomes, expenses, budgets and goals. SQLite handles this load without administration, and the ORM isolates the application from the database engine. A move to PostgreSQL or MySQL would change only the connection string and the data types that need to be adjusted.

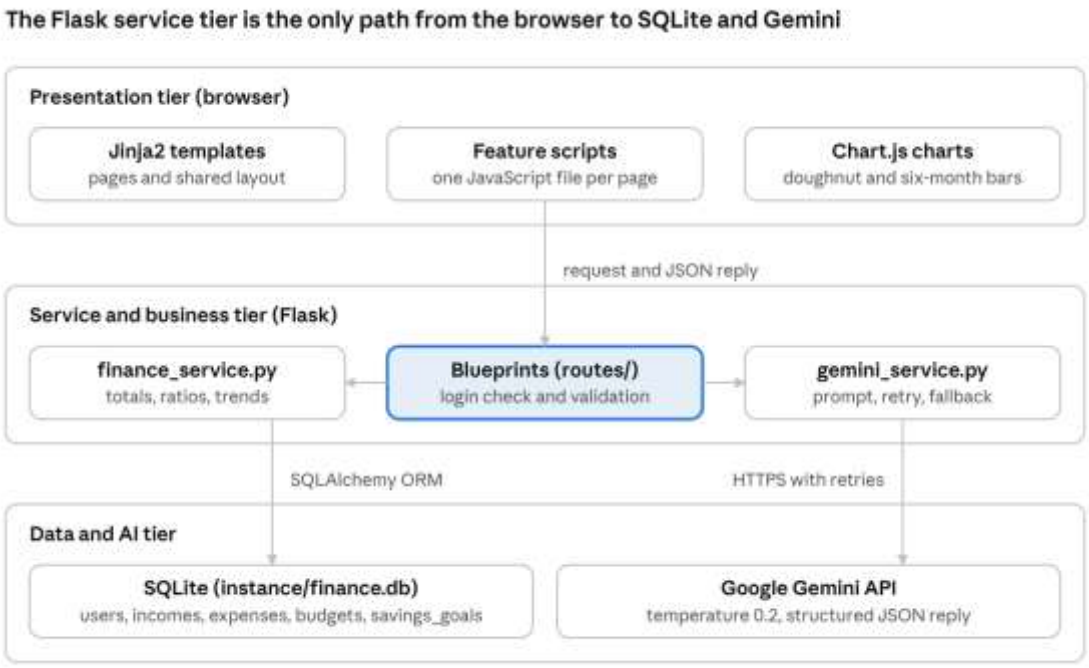
Werkzeug password hashing. The hash string stores the algorithm, a per-password random salt and the derived key in one value. The login route compares the submitted password with that value through check_password_hash, so the plain password is never written to the database or to a log [6], [12].

Vanilla JavaScript. The interface consists of a small number of pages with forms, tables and two charts. A front-end framework would add a build chain without a corresponding benefit, so the code uses the browser's fetch function directly [15].

google-genai SDK. The application depends on the current official SDK for Gemini rather than on direct HTTP calls. This keeps authentication, request formatting and error types inside a maintained library.

7. System Architecture

The application is a modular monolith with a three-tier layered structure. The presentation tier runs in the browser, the service tier runs in the Flask process, and the persistence and intelligence tier consists of the SQLite database and the Gemini API. Figure 1 shows the tiers and the direction of each request.



three-tier architecture · 3 tiers, 5 connections

Figure 1: Three-tier architecture of the Personal Finance Advisor Bot. The browser reaches the blueprints only through JSON requests. The blueprints call finance_service.py for calculations and gemini_service.py for advice, and only these two services reach SQLite and the Gemini API.

7.1 Three-Tier Architecture Description

Presentation tier. The Jinja2 templates (base.html, index.html, login.html, register.html, dashboard.html, privacy.html and terms.html) provide the page structure. One JavaScript file per feature (app.js, dashboard.js, income.js, expenses.js, budgets.js, goals.js and ai-advisor.js) calls the JSON API with fetch, writes the results into the page and updates the Chart.js charts. The browser holds no business rules except basic form checks, and the server repeats every validation.

Service and business tier. The routes package contains the blueprints. Each blueprint authenticates the request, reads and validates the input, calls the service functions and returns JSON. The services package holds the logic that is independent of HTTP. finance_service.py performs the arithmetic, the INR formatting and the six-month aggregation. gemini_service.py builds the prompt, calls the SDK, retries transient failures, extracts and validates the JSON, and produces the fallback analysis.

Data and AI tier. The models package defines the five SQLAlchemy entities. SQLite stores them in instance/finance.db, which is excluded from version control. The Gemini API is an external service reached over HTTPS through the SDK with the key taken from the environment.

7.2 Architectural Workflow and Data Flow

The application factory in app.py creates the Flask object, loads config.py, initialises the database, enables CORS for the API, registers the blueprints and installs the error handlers. The handlers return JSON for API paths so that the front end always receives a parseable body.

The data flow for an AI analysis request has seven steps.

1. The user selects a month on the dashboard and chooses the analysis action. ai-advisor.js sends POST /api/ai/analyze with the month.
2. The login_required decorator in routes/**init**.py checks the session. A missing session returns an authentication error and no further work is done.
3. The ai blueprint calls finance_service to read the user's incomes, expenses, budgets and goals for the month and to compute the totals, the category distribution and the budget statuses.
4. gemini_service builds the prompt from these figures and calls client.models.generate_content with a temperature of 0.2.
5. On an HTTP 503 response or a quota error, the call is retried with an increasing delay. If all attempts fail, or if the reply cannot be parsed into the required keys, the service calls the fallback function instead.
6. The route adds the source of the analysis and the disclaimer to the result and returns it as JSON.
7. ai-advisor.js renders the summary, insights, overspending categories, recommendations and reductions in the advisory panel.

The other routes follow the same pattern without steps 4 and 5. A create, update or delete request is authenticated, validated, applied to the database through the ORM and answered with the saved record or a status message.

7.3 Repository Structure

```
personal-finance/
├── app.py              # Application factory, blueprint registration, error handlers
├── config.py           # Environment configuration, SQLite path, Gemini model
settings
├── requirements.txt    # Flask, Flask-SQLAlchemy, Flask-CORS, python-dotenv, google-
genai, Werkzeug
├── seed.py             # Demonstration data (demo@financebot.local)
├── test_app.py         # Automated unit and integration tests
├── .env.example        # Template of required environment variables
├── .gitignore          # Excludes .env, instance/*.db and venv/
├── models/             # user.py, income.py, expense.py, budget.py, savings_goal.py
├── routes/             # pages.py, auth.py, dashboard.py, income.py, expenses.py,
│                       # budgets.py, goals.py, ai.py, and login_required in
__init__.py
├── services/           # finance_service.py, gemini_service.py
├── templates/          # base, index, login, register, dashboard, privacy, terms
└── static/             # favicon.svg, css/style.css, js/*.js
```

8. System Modules

The application is divided into eight functional modules. Each module has its own blueprint or service function, its own JavaScript file where it has a page, and a defined set of API routes. Table 3 summarises them, and the sub-sections describe each module in turn.

Table 3: Summary of functional modules

No.	Module	Purpose	Main implementation
1	Authentication and session management	Create accounts, verify credentials, keep a session and protect routes	routes/auth.py, login_required in routes/**init**.py, models/user.py
2	Financial overview dashboard	Show monthly totals, savings rate, category distribution, budget status and six-month cash flow	routes/dashboard.py, services/finance_service.py, static/js/dashboard.js
3	Income ledger	Record and maintain income entries by source and date	routes/income.py, models/income.py, static/js/income.js
4	Categorised expense tracker	Record and maintain expense entries in nine categories	routes/expenses.py, models/expense.py, static/js/expenses.js
5	Category budget planner and	Set monthly limits per category and flag warning and exceeded	routes/budgets.py, models/budget.py,

No.	Module	Purpose	Main implementation
	variance alerting	budgets	static/js/budgets.js
6	Savings targets and quick deposit	Track goals, progress and target dates; add deposits	routes/goals.py, models/savings_goal.py, static/js/goals.js
7	Gemini AI advisory engine and rule-based fallback	Produce structured advice from the monthly figures	routes/ai.py, services/gemini_service.py, static/js/ai-advisor.js
8	Compliance, privacy policy and terms of service	State data handling and the advisory-only nature of the output	routes/pages.py, templates/privacy.html, templates/terms.html

8.1 Module 1: Authentication and Session Management

A visitor registers with a name, an email address and a password. The server converts the email address to lower case, checks that it is not already registered, and stores only the salted hash of the password. At login the submitted password is verified against the stored hash. On success the user identifier is written to the Flask session, which is a signed cookie, and later requests are identified by it. The login_required decorator rejects an unauthenticated API request with a 401 response and redirects an unauthenticated page request to the login page.

```
# routes/auth.py (registration and login, condensed)
from werkzeug.security import generate_password_hash, check_password_hash

@auth_bp.post("/register")
def register():
    data = request.get_json(silent=True) or {}
    name = (data.get("name") or "").strip()
    email = (data.get("email") or "").strip().lower()
    password = data.get("password") or ""
    if not name or "@" not in email or len(password) < MIN_PASSWORD_LENGTH:
        return jsonify({"error": "Provide a name, a valid email and a longer
password."}), 400
    if User.query.filter_by(email=email).first():
        return jsonify({"error": "This email is already registered."}), 409
    user = User(name=name, email=email,
                password_hash=generate_password_hash(password))
    db.session.add(user)
    db.session.commit()
    return jsonify({"message": "Account created."}), 201

@auth_bp.post("/login")
def login():
    data = request.get_json(silent=True) or {}
    email = (data.get("email") or "").strip().lower()
    user = User.query.filter_by(email=email).first()
    if user is None or not check_password_hash(user.password_hash,
data.get("password") or ""):
```

```
        return jsonify({"error": "Invalid email or password."}), 401
    session.clear()
    session["user_id"] = user.id
    return jsonify({"id": user.id, "name": user.name, "email": user.email})
```

```
# routes/__init__.py
from functools import wraps
from flask import session, redirect, url_for, request, jsonify

def login_required(view):
    @wraps(view)
    def wrapped(*args, **kwargs):
        if "user_id" not in session:
            if request.path.startswith("/api/"):
                return jsonify({"error": "Authentication required."}), 401
            return redirect(url_for("pages.login_page"))
        return view(*args, **kwargs)
    return wrapped
```

The login route returns the same message for an unknown email and a wrong password, so the response does not reveal which email addresses are registered. The session is cleared before the identifier is written, which prevents an earlier session from being carried into the new login. GET /api/auth/me returns the current user's name and email and is used by the front end to display the account in the header.

8.2 Module 2: Financial Overview Dashboard

The dashboard is the home screen after login. For the selected month it shows four summary cards (total income, total expenses, net savings and savings rate), a doughnut chart of expenses by category, a six-month bar chart of income, expenses and net savings, the budget status of each category, and the progress of the savings goals. A month selector sends GET /api/dashboard?month=YYYY-MM, and the server returns one JSON object that contains all of these values, so the page needs a single request. The formulas are given in Chapter 9.

8.3 Module 3: Income Ledger

The income ledger lists the entries of the selected month with amount, source, date and description. The form accepts a positive amount, a source name such as Primary Salary, Freelance or Dividend, a date and an optional description. The server rejects a missing amount, a zero or negative amount, and an unparseable date. Entries can be edited and deleted. An update or delete for an identifier that belongs to another user returns a not-found response, which does not disclose that the record exists.

8.4 Module 4: Categorised Expense Tracker

Each expense has an amount, a category, a date and an optional description. The category must be one of the nine standard values, and any other value is rejected with a validation error.

- **Food:** groceries and daily provisions
- **Rent:** apartment and housing
- **Transport:** metro, fuel and commute
- **Education:** courses, books and certifications

- **Healthcare:** medical expenses, prescriptions and dental care
- **Entertainment:** streaming and outings
- **Shopping:** apparel and hardware
- **Utilities:** electricity, water, broadband and gas
- **Other:** incidental and unclassified expenses

The list view can be filtered by month and by category. The expense list is the source of the category distribution on the dashboard and of the amounts compared with the budgets.

8.5 Module 5: Category Budget Planner and Variance Alerting

A budget is a monthly spending limit for one category. The planner stores at most one budget for each combination of user, month and category: saving a limit for an existing combination updates it, and saving it for a new combination creates it. For every budget the server calculates the amount spent, the remaining amount and the utilisation percentage, and assigns a status.

- **On track:** utilisation of 80% or less
- **Warning:** utilisation above 80% and up to 100%
- **Exceeded:** utilisation above 100%

The status is returned with the budget and with the dashboard, and the interface marks it by text and by colour so that the meaning does not depend on colour alone. The calculation is given in Section 9.2.

8.6 Module 6: Savings Target Milestones and Quick Deposit Workflow

A savings goal has a name, a target amount, a current amount and an optional target date. The goals page shows each goal with a progress bar, the percentage achieved, the amount remaining and the days left until the target date. The quick deposit action sends POST /api/goals/<id>/deposit with a positive amount, which the server adds to the current amount of that goal. The updated goal is returned, and the progress bar is redrawn without a reload. Deposits are recorded against the goal and are independent of the expense ledger, so a transfer to a goal does not appear as an expense.

8.7 Module 7: Gemini AI Financial Advisory Engine and Rule-Based Fallback

The advisory module takes the aggregated figures for a month and returns structured advice. The request is processed by gemini_service.py in four stages: prompt construction, model call with retry, extraction and validation of the JSON reply, and substitution of the fallback analysis when any earlier stage fails. The module is described in Chapter 12.

8.8 Module 8: Compliance, Privacy Policy and Terms of Service

The /privacy and /terms pages are public. The Privacy Policy states which data is collected (name, email address, ledger entries, budgets and goals), where it is stored, that passwords are kept only as hashes, and that the monthly figures are sent to the Gemini API when the user requests an analysis. The Terms of Service state that the output is informational, that it is not professional financial, tax or investment advice, and that the user is responsible for decisions taken on the basis of it. The same disclaimer is attached to every analysis returned by the API.

9. Mathematical and Financial Calculations

All figures shown by the application are calculated on the server in finance_service.py. The browser only formats and draws them. This section gives the formulas and the code that implements them. Amounts are in INR, and a month is identified by a string in the form YYYY-MM.

9.1 Total Income, Total Expenses, Net Savings and Savings Rate

For a selected month, let each income entry have an amount and each expense entry have an amount. The totals are the sums of the amounts of the entries dated in that month.

```
TI = \sum_{i=1}^{n} \text{income}_i \qquad
TE = \sum_{j=1}^{m} \text{expense}_j
```

Net savings is the difference between the two totals. The savings rate expresses net savings as a percentage of total income, and it is defined as zero when total income is zero, so that the division is never performed with a zero divisor. The expense ratio is the complementary measure.

```
NS = TI - TE
\qquad
SR = \frac{NS}{TI} \times 100
\qquad
ER = \frac{TE}{TI} \times 100
```

For the demonstration data in Chapter 15, TI is ₹1,08,000.00 and TE is ₹64,500.00. Therefore NS = ₹43,500.00, SR = 43,500 ÷ 1,08,000 × 100 = 40.28% and ER = 59.72%.

The implementation uses a single aggregate query for each total and rounds the result to two decimal places.

```
# services/finance_service.py (condensed)
from sqlalchemy import func

def _total(model, user_id, start, end):
    value = (db.session.query(func.coalesce(func.sum(model.amount), 0.0))
             .filter(model.user_id == user_id,
                     model.date >= start, model.date < end)
             .scalar())
    return round(float(value), 2)

def month_summary(user_id, month):
    start, end = month_bounds(month)          # first day, first day of next month
    total_income = _total(Income, user_id, start, end)
    total_expenses = _total(Expense, user_id, start, end)
    net_savings = round(total_income - total_expenses, 2)
    savings_rate = round(net_savings / total_income * 100, 2) if total_income > 0
else 0.0
    return {
        "month": month,
        "total_income": total_income,
        "total_expenses": total_expenses,
        "net_savings": net_savings,
        "savings_rate": savings_rate,
    }
```

Amounts are formatted for display by a function that groups digits in the Indian manner: the last three digits form one group and the remaining digits are grouped in pairs.

```
def format_inr(value):
    """150000 -> ₹1,50,000.00 ; 108000 -> ₹1,08,000.00 ; -2500.5 -> -₹2,500.50"""
    negative = value < 0
    whole, frac = f"{abs(value):.2f}".split(".")
    if len(whole) > 3:
        head, tail = whole[:-3], whole[-3:]
        groups = []
        while len(head) > 2:
            groups.insert(0, head[-2:])
            head = head[:-2]
        if head:
            groups.insert(0, head)
        whole = ",".join(groups + [tail])
    return f"{'-' if negative else ''}₹{whole}.{frac}"
```

9.2 Budget Variance, Category Distribution and Threshold Detection

For a category c with expenses S_c in the month and a budget limit L_c, the variance is the spent amount minus the limit, the remaining amount is its negative, and the utilisation is the spent amount as a percentage of the limit.

```
V_c = S_c - L_c
\qquad
R_c = L_c - S_c
\qquad
U_c = \frac{S_c}{L_c} \times 100
```

The status is derived from the utilisation with two fixed thresholds.

```
\text{status}_c =
\begin{cases}
\text{Exceeded} & U_c > 100 \\
\text{Warning} & 80 < U_c \le 100 \\
\text{On track} & U_c \le 80
\end{cases}
```

The distribution of expenses across categories is the share of each category in total expenses. It is the quantity drawn in the doughnut chart.

```
D_c = \frac{S_c}{TE} \times 100 \qquad \sum_{c=1}^{9} D_c = 100
```

Two budgets of the demonstration data illustrate the thresholds. The Entertainment limit is ₹3,000.00 and ₹3,850.00 was spent, so V = +₹850.00 and U = 128.33%, which is Exceeded. The Shopping limit is ₹6,500.00 and ₹6,400.00 was spent, so R = ₹100.00 and U = 98.46%, which is Warning.

```
def budget_status(limit, spent):
    utilisation = round(spent / limit * 100, 2) if limit > 0 else 0.0
    if utilisation > 100:
        status = "exceeded"
    elif utilisation > 80:
        status = "warning"
    else:
        status = "on_track"
    return {"limit": limit, "spent": spent,
```

```
            "remaining": round(limit - spent, 2),
            "variance": round(spent - limit, 2),
            "utilisation": utilisation, "status": status}
```

9.3 Six-Month Rolling Cash-Flow Time Series

The bar chart on the dashboard shows income, expenses and net savings for the selected month and the five months before it. The server first builds the list of six month keys, then runs one grouped query per table, and finally merges the results so that a month without entries appears with zero values and the chart axis has no gaps.

```
def last_six_months(anchor):                        # anchor = "2026-10"
    year, mon = map(int, anchor.split("-"))
    months = []
    for offset in range(5, -1, -1):
        m, y = mon - offset, year
        while m <= 0:
            m, y = m + 12, y - 1
        months.append(f"{y:04d}-{m:02d}")
    return months                                   # oldest first
```

```
-- Monthly expense totals for one user over the window (SQLite)
SELECT strftime('%Y-%m', date) AS ym,
       ROUND(SUM(amount), 2)   AS total
FROM   expenses
WHERE  user_id = :uid
  AND  date >= :window_start
  AND  date <  :window_end
GROUP  BY ym
ORDER  BY ym;
```

The same query is run against the incomes table. For each month key m in the window, the series values are TI(m), TE(m) and NS(m) = TI(m) − TE(m). The query reads at most six months of one user's rows and is served by the index on user_id and date described in Chapter 10.

9.4 The 50/30/20 Allocation Used by the Fallback Engine

The deterministic fallback in Section 12.3 compares the month with the 50/30/20 rule of thumb described by Warren and Tyagi [11]: about 50% of after-tax income for needs, 30% for wants and 20% for savings. The application groups its categories as follows.

- **Needs:** Rent, Food, Transport, Utilities, Healthcare and Education
- **Wants:** Shopping, Entertainment and Other

```
P_{\text{needs}} = \frac{\sum S_c^{\text{needs}}}{TI} \times 100
\qquad
P_{\text{wants}} = \frac{\sum S_c^{\text{wants}}}{TI} \times 100
\qquad
P_{\text{savings}} = SR
```

For the demonstration data the needs total ₹53,050.00 (49.12% of income), the wants total ₹11,450.00 (10.60%) and the savings rate is 40.28%. Needs are below the 50% guide, wants are below 30%, and savings are above 20%. The fallback therefore reports a healthy allocation and directs its suggestions at the categories that exceed their budgets.

10. Database Design and Data Modelling

The persistence layer uses SQLite through Flask-SQLAlchemy [3], [4], [5]. The five tables are created at start-up from the model classes with db.create_all(). Each ledger table carries a user_id column, so the ownership of every row is explicit and every query can be restricted to the signed-in user.

10.1 Entity-Relationship Overview

The schema has one parent entity, users, and four child entities. Each relationship is one-to-many, and a row of a child table belongs to exactly one user.

- users (1) to incomes (N): a user records many income entries.
- users (1) to expenses (N): a user records many expense entries.
- users (1) to budgets (N): a user holds many budgets, at most one for each month and category.
- users (1) to savings_goals (N): a user keeps many goals.

The child tables are not related to each other. A budget is compared with the sum of the expenses of the same user, month and category at query time, and no foreign key links the two tables. This avoids duplicated totals that could become inconsistent when an expense is edited. Deleting a user removes the user's rows in all four child tables through the cascade rule.

```
# models/budget.py (condensed)
class Budget(db.Model):
    __tablename__ = "budgets"
    __table_args__ = (
        db.UniqueConstraint("user_id", "month", "category",
name="uq_budget_user_month_cat"),
    )
    id       = db.Column(db.Integer, primary_key=True)
    user_id  = db.Column(db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"),
nullable=False, index=True)
    month    = db.Column(db.String(7), nullable=False)        # "YYYY-MM"
    category = db.Column(db.String(50), nullable=False)
    limit    = db.Column(db.Float, nullable=False)
```

10.2 Table: users

Table 4: Structure of the users table

Column	Type	Constraints	Description
id	Integer	Primary key	Unique identifier of the user
name	String(100)	Not null	Full name entered at registration
email	String(120)	Unique, not null, indexed	Login identifier, stored in lower case
password_hash	String(255)	Not null	Salted hash created by Werkzeug; the password itself is never stored
created_at	DateTime	Default: current	Time of registration

Column	Type	Constraints	Description
		UTC time	

10.3 Table: incomes

Table 5: Structure of the incomes table

Column	Type	Constraints	Description
id	Integer	Primary key	Unique identifier of the income entry
user_id	Integer	Foreign key to users.id, not null, indexed	Owner of the entry
amount	Float	Not null, greater than zero	Amount received in INR
source	String(100)	Not null	Origin of the income, for example Primary Salary, Freelance or Dividend
date	Date	Not null	Date on which the amount was received
description	String(255)	Nullable	Optional note

10.4 Table: expenses

Table 6: Structure of the expenses table

Column	Type	Constraints	Description
id	Integer	Primary key	Unique identifier of the expense entry
user_id	Integer	Foreign key to users.id, not null, indexed	Owner of the entry
amount	Float	Not null, greater than zero	Amount spent in INR
category	String(50)	Not null; one of the nine standard categories	Food, Rent, Transport, Education, Healthcare, Entertainment, Shopping, Utilities or Other
date	Date	Not null	Date of the expense
description	String(255)	Nullable	Optional note, for example the merchant or the item

10.5 Table: budgets

Table 7: Structure of the budgets table

Column	Type	Constraints	Description
id	Integer	Primary key	Unique identifier of the budget
user_id	Integer	Foreign key to users.id, not null, indexed	Owner of the budget
month	String(7)	Not null	Month to which the limit applies, in the form YYYY-MM
category	String(50)	Not null; one of the nine standard categories	Budgeted category
limit	Float	Not null, greater than zero	Maximum planned spending for the month in INR

The combination of user_id, month and category is unique. The budget route uses this rule to implement the upsert: it looks up the combination and updates the limit if a row exists, and inserts a row otherwise.

10.6 Table: savings_goals

Table 8: Structure of the savings_goals table

Column	Type	Constraints	Description
id	Integer	Primary key	Unique identifier of the goal
user_id	Integer	Foreign key to users.id, not null, indexed	Owner of the goal
name	String(100)	Not null	Name of the goal, for example Emergency Reserve
target_amount	Float	Not null, greater than zero	Amount to be saved in INR
current_amount	Float	Not null, default 0	Amount saved so far; increased by the quick deposit action
target_date	Date	Nullable	Date by which the target is to be reached

10.7 Equivalent SQL Definition

The following statements are the SQLite equivalent of the models. The limit column is a reserved word in SQL and is therefore written in quotation marks. The CHECK constraints document the valid ranges, which the route layer enforces before any row is written.

```
CREATE TABLE users (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    name          VARCHAR(100) NOT NULL,
```

```
    email         VARCHAR(120) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE incomes (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    amount      FLOAT   NOT NULL CHECK (amount > 0),
    source      VARCHAR(100) NOT NULL,
    date        DATE    NOT NULL,
    description VARCHAR(255)
);

CREATE TABLE expenses (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id     INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    amount      FLOAT   NOT NULL CHECK (amount > 0),
    category    VARCHAR(50) NOT NULL CHECK (category IN (
                  'Food','Rent','Transport','Education','Healthcare',
                  'Entertainment','Shopping','Utilities','Other')),
    date        DATE    NOT NULL,
    description VARCHAR(255)
);

CREATE TABLE budgets (
    id       INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id  INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    month    VARCHAR(7)  NOT NULL,
    category VARCHAR(50) NOT NULL,
    "limit"  FLOAT NOT NULL CHECK ("limit" > 0),
    UNIQUE (user_id, month, category)
);

CREATE TABLE savings_goals (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id        INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name           VARCHAR(100) NOT NULL,
    target_amount  FLOAT NOT NULL CHECK (target_amount > 0),
    current_amount FLOAT NOT NULL DEFAULT 0,
    target_date    DATE
);

-- Supports the monthly and six-month aggregations of Chapter 9
CREATE INDEX ix_incomes_user_date  ON incomes  (user_id, date);
CREATE INDEX ix_expenses_user_date ON expenses (user_id, date);
```

10.8 Data Isolation

Every route that reads or changes a record includes the owner in the query. A request for an identifier that exists but belongs to another user returns the same not-found response as a request for an identifier that does not exist.

```
expense = Expense.query.filter_by(id=expense_id,
user_id=session["user_id"]).first_or_404()
```

11. REST API Specification

The server exposes two kinds of routes. Page routes return server-rendered HTML, and API routes under the /api prefix accept and return JSON. Each feature is a Flask blueprint registered in the application factory. Table 9 lists every route of the application.

Table 9: Route registry with methods, authentication and purpose

Method	Route	Authentication	Purpose
GET	/	Public	Landing page
GET	/login	Public	Login page
GET	/register	Public	Registration page
GET	/dashboard	Session	Dashboard page; redirects to /login without a session
GET	/privacy	Public	Privacy Policy
GET	/terms	Public	Terms of Service
POST	/api/auth/register	Public	Create an account; stores the password hash
POST	/api/auth/login	Public	Verify credentials and start a session
POST	/api/auth/logout	Session	End the session
GET	/api/auth/me	Session	Return the name and email of the signed-in user
GET	/api/dashboard?month=YYYY-MM	Session	Return totals, savings rate, category distribution, budget statuses, goals and the six-month series
GET	/api/income	Session	List income entries, optionally filtered by month
POST	/api/income	Session	Create an income entry
PUT	/api/income/<id>	Session	Update an income entry owned by the user
DELETE	/api/income/<id>	Session	Delete an income entry owned by the user
GET	/api/expenses	Session	List expenses, optionally filtered by month and category
POST	/api/expenses	Session	Create an expense; the category is validated

Method	Route	Authentication	Purpose
PUT	/api/expenses/<id>	Session	Update an expense owned by the user
DELETE	/api/expenses/<id>	Session	Delete an expense owned by the user
GET	/api/budgets	Session	List budgets with spent amount, utilisation and status for a month
POST	/api/budgets	Session	Create or update (upsert) the limit for a month and category
PUT	/api/budgets/<id>	Session	Change the limit of a budget
DELETE	/api/budgets/<id>	Session	Delete a budget
GET	/api/goals	Session	List savings goals with progress
POST	/api/goals	Session	Create a savings goal
PUT	/api/goals/<id>	Session	Update the name, target, saved amount or target date
DELETE	/api/goals/<id>	Session	Delete a goal
POST	/api/goals/<id>/deposit	Session	Add a deposit to the current amount of a goal
POST	/api/ai/analyze	Session	Return the AI financial analysis for a month, from Gemini or the fallback engine

11.1 Conventions

- **Format.** Requests with a body use Content-Type application/json. Every API response, including errors, is JSON.
- **Status codes.** 200 for a successful read or update, 201 for a created record, 400 for failed validation, 401 when no valid session exists, 404 when a record does not exist or is not owned by the user, 409 for a duplicate email, and 500 for an unexpected server error.
- **Errors.** An error body has the form {"error": "message"}. The registered error handlers return this form for any path that starts with /api, so the front end does not receive an HTML error page.
- **Months and dates.** A month is YYYY-MM and a date is an ISO 8601 date (YYYY-MM-DD). Amounts are JSON numbers in INR.
- **Ownership.** The user identifier is read from the session and is never accepted from the request body or the query string.

11.2 Examples

The request and response below create an expense and show the shape of a validation error.

```
POST /api/expenses
{
  "amount": 3850.00,
  "category": "Entertainment",
  "date": "2026-10-12",
  "description": "Streaming subscriptions and an outing"
}

201 Created
{
  "id": 41,
  "amount": 3850.0,
  "category": "Entertainment",
  "date": "2026-10-12",
  "description": "Streaming subscriptions and an outing"
}

POST /api/expenses   (category not in the nine standard values)
400 Bad Request
{ "error": "Category must be one of: Food, Rent, Transport, Education, Healthcare,
Entertainment, Shopping, Utilities, Other." }
```

The budget status object returned for a month has the following form. The values are those of the demonstration data in Chapter 15.

```
GET /api/budgets?month=2026-10
[
  { "category": "Entertainment", "limit": 3000.0, "spent": 3850.0,
    "remaining": -850.0, "utilisation": 128.33, "status": "exceeded" },
  { "category": "Shopping",      "limit": 6500.0, "spent": 6400.0,
    "remaining": 100.0,  "utilisation": 98.46,  "status": "warning" }
]
```

The response of POST /api/ai/analyze is described with its schema in Section 12.2. The server adds the key source, with the value gemini or fallback, so that the interface can state which engine produced the text.

12. Gemini AI Integration and Prompt Engineering

The advisory engine is implemented in services/gemini_service.py and is called by POST /api/ai/analyze. The design follows one rule: the application computes every number, and the model only comments on numbers that it has been given. The model is called through the official google-genai SDK [8], [9], with the model name read from the configuration (gemini-3.8-flash) and the temperature fixed at 0.2 so that repeated requests for the same month give similar answers.

12.1 Structured Prompt Architecture with INR Financial Context

The prompt has six parts, written in a fixed order.

1. **Role.** The model is told that it is a personal finance adviser for an individual in India and that all amounts are in Indian Rupees.
2. **Month summary.** Month, total income, total expenses, net savings and savings rate, as computed in Section 9.1.

3. **Expenses by category.** For each of the nine categories, the amount and its percentage of income.
4. **Budgets.** For each budget, the limit, the amount spent, the utilisation and the status from Section 9.2.
5. **Goals.** For each goal, the saved amount, the target amount, the progress percentage and the target date.
6. **Task and output rules.** The six required sections, an instruction to use only the supplied figures, and an instruction to return JSON with the stated keys and nothing else.

The prompt contains category names and aggregated amounts only. The user's name, email address and the free-text descriptions of individual entries are not included. The builder is a plain function, so it can be tested without a network call.

```
# services/gemini_service.py (prompt builder, condensed)
ROLE = (
    "You are a personal finance adviser for an individual in India. "
    "All amounts are in Indian Rupees (INR, ₹). Use only the figures given below. "
    "Do not invent income, expenses, interest rates or products."
)

def build_prompt(summary, by_category, budgets, goals):
    cat_lines = "\n".join(
        f"- {c['category']}: {format_inr(c['amount'])} ({c['percent_of_income']:.2f}%
of income)"
        for c in by_category)
    budget_lines = "\n".join(
        f"- {b['category']}: limit {format_inr(b['limit'])}, spent
{format_inr(b['spent'])}, "
        f"{b['utilisation']:.2f}% used, status {b['status']}" for b in budgets) or "-
none set"
    goal_lines = "\n".join(
        f"- {g['name']}: {format_inr(g['current_amount'])} of
{format_inr(g['target_amount'])} "
        f"({g['progress']:.2f}%), target date {g['target_date'] or 'not set'}" for g
in goals) or "- none set"
    return f"""{ROLE}

MONTH: {summary['month']}
Total income: {format_inr(summary['total_income'])}
Total expenses: {format_inr(summary['total_expenses'])}
Net savings: {format_inr(summary['net_savings'])}
Savings rate: {summary['savings_rate']:.2f}%

EXPENSES BY CATEGORY
{cat_lines}

BUDGETS
{budget_lines}

SAVINGS GOALS
{goal_lines}

TASK
```

```
Return a single JSON object with exactly these keys:
monthly_financial_summary (string), spending_insights (list of strings),
overspending_categories (list of objects), budget_recommendations (list of objects),
saving_suggestions (list of objects), unnecessary_expense_reductions (list of
objects),
disclaimer (string). Return JSON only, without Markdown fences or commentary."""
```

12.2 Output JSON Schema Enforcement and Extraction Logic

The prompt asks for JSON, but a language model can still wrap the object in text or in a Markdown fence, and it can omit a key. The application therefore treats the reply as untrusted input [14] and accepts it only after parsing and checking. The required structure is shown below with values from the demonstration data.

```
{
  "monthly_financial_summary": "Income of ₹1,08,000.00 against expenses of ₹64,500.00
leaves ₹43,500.00, a savings rate of 40.28%.",
  "spending_insights": ["Rent is the largest category at 22.22% of income.",
                        "Food is the second largest at 12.92% of income."],
  "overspending_categories": [
    {"category": "Entertainment", "spent": 3850.0, "limit": 3000.0,
     "excess": 850.0, "comment": "The budget is exceeded by 28.33%."}],
  "budget_recommendations": [
    {"category": "Shopping", "recommended_limit": 6000.0,
     "reason": "Spending is at 98.46% of the current limit."}],
  "saving_suggestions": [
    {"suggestion": "Move a fixed amount to the Himalayan Trek goal at the start of
the month.",
     "estimated_monthly_amount": 3000.0}],
  "unnecessary_expense_reductions": [
    {"category": "Entertainment", "suggested_reduction": 850.0,
     "action": "Review overlapping streaming subscriptions."}],
  "disclaimer": "This analysis is informational and is not professional financial
advice."
}
```

The function extract_json() applies three strategies in order. It first tries to parse the whole reply. If that fails, it looks for a fenced block and parses its content. If that also fails, it parses the text between the first opening brace and the last closing brace. The result is accepted only if it is a JSON object that contains all required keys. The route then replaces the disclaimer with the fixed application text, so the disclaimer is present even when the model omits it.

```
REQUIRED_KEYS = ("monthly_financial_summary", "spending_insights",
"overspending_categories",
                 "budget_recommendations", "saving_suggestions",
                 "unnecessary_expense_reductions", "disclaimer")
FENCE = re.compile(r"```(?:json)?\s*(\{.*?\})\s*```", re.DOTALL)

def extract_json(text):
    candidates = [text]
    m = FENCE.search(text or "")
    if m:
        candidates.append(m.group(1))
    if text and "{" in text and "}" in text:
        candidates.append(text[text.index("{"): text.rindex("}") + 1])
```

```
    for candidate in candidates:
        try:
            data = json.loads(candidate)
        except (TypeError, ValueError):
            continue
        if isinstance(data, dict) and all(k in data for k in REQUIRED_KEYS):
            return data
    return None
```

12.3 Transient Error Handling and the Deterministic Fallback Engine

Calls to a hosted model can fail for reasons that are outside the application, such as an HTTP 503 Service Unavailable response, a quota limit or a network interruption [13]. The call is made at most three times. After a transient failure the code waits for one second and then for two seconds, which is an exponential delay with a base of one second. A failure that is not transient, for example an invalid API key, is not retried. If no key is configured, the Gemini call is skipped and the fallback is used immediately.

```
RETRY_DELAYS = (1.0, 2.0)           # two retries after the first attempt

def _is_transient(exc):
    text = str(exc).upper()
    return any(token in text for token in ("503", "UNAVAILABLE", "429",
"RESOURCE_EXHAUSTED",
                                           "TIMEOUT", "CONNECTION"))

def analyze(summary, by_category, budgets, goals):
    api_key = current_app.config.get("GEMINI_API_KEY")
    if api_key:
        client = genai.Client(api_key=api_key)
        prompt = build_prompt(summary, by_category, budgets, goals)
        for attempt in range(len(RETRY_DELAYS) + 1):
            try:
                response = client.models.generate_content(
                    model=current_app.config["GEMINI_MODEL"],       # gemini-3.8-
flash
                    contents=prompt,
                    config=types.GenerateContentConfig(temperature=0.2))
                result = extract_json(response.text)
                if result:
                    result["disclaimer"] = DISCLAIMER
                    result["source"] = "gemini"
                    return result
                break                                   # unusable reply: do not
retry
            except Exception as exc:
                if not _is_transient(exc) or attempt == len(RETRY_DELAYS):
                    break
                time.sleep(RETRY_DELAYS[attempt])
    return _build_fallback_analysis(summary, by_category, budgets, goals)
```

The fallback engine is a deterministic function of the same inputs. It uses the 50/30/20 allocation of Section 9.4, the budget statuses of Section 9.2 and the goal progress, and it produces the same keys as the Gemini reply so that the front end needs no special case. Because it uses no network call and no random choice, it returns the same text for the same data and cannot fail because of an external service.

```
NEEDS = {"Rent", "Food", "Transport", "Utilities", "Healthcare", "Education"}
WANTS = {"Shopping", "Entertainment", "Other"}

def _build_fallback_analysis(summary, by_category, budgets, goals):
    income = summary["total_income"]
    pct = lambda part: round(part / income * 100, 2) if income > 0 else 0.0
    needs = sum(c["amount"] for c in by_category if c["category"] in NEEDS)
    wants = sum(c["amount"] for c in by_category if c["category"] in WANTS)
    over = [b for b in budgets if b["status"] == "exceeded"]
    near = [b for b in budgets if b["status"] == "warning"]
    top = max(by_category, key=lambda c: c["amount"], default=None)

    insights = [f"Needs use {pct(needs):.2f}% of income against a guide of 50%.",
                f"Wants use {pct(wants):.2f}% of income against a guide of 30%.",
                f"Savings are {summary['savings_rate']:.2f}% of income against a
guide of 20%."]
    if top:
        insights.append(f"{top['category']} is the largest category at
{pct(top['amount']):.2f}% of income.")

    return {
        "monthly_financial_summary":
            f"Income {format_inr(income)}, expenses
{format_inr(summary['total_expenses'])}, "
            f"net savings {format_inr(summary['net_savings'])}.",
        "spending_insights": insights,
        "overspending_categories": [
            {"category": b["category"], "spent": b["spent"], "limit": b["limit"],
             "excess": round(b["spent"] - b["limit"], 2),
             "comment": f"Budget exceeded at {b['utilisation']:.2f}% utilisation."}
for b in over],
        "budget_recommendations": [
            {"category": b["category"], "recommended_limit": b["limit"],
             "reason": f"Utilisation is {b['utilisation']:.2f}%; keep the next month
below the limit."}
            for b in over + near],
        "saving_suggestions": [
            {"suggestion": "Transfer savings to goals on the day income is
received.",
             "estimated_monthly_amount": round(income * 0.20, 2)}],
        "unnecessary_expense_reductions": [
            {"category": b["category"], "suggested_reduction": round(b["spent"] -
b["limit"], 2),
             "action": "Reduce spending to the budget limit."} for b in over],
        "disclaimer": DISCLAIMER,
        "source": "fallback",
    }
```

The automated test for this module (TC-10) checks that a structurally valid analysis is returned and that the disclaimer is present, and that the fallback path produces a valid result when the Gemini call is not available.

13. Frontend UI/UX Design System

The interface is a custom design system written in plain CSS (static/css/style.css) and plain JavaScript. It is intended for repeated, practical use by a person who enters and reviews figures, so the rules favour legibility, alignment and predictable controls over decoration.

13.1 Design Principles (Anti-Vibe-Coding)

The term anti-vibe-coding is used in this project for the practice of choosing each visual rule on purpose and writing it down, instead of accepting the default styling that code generators and templates tend to produce. The rules are listed below.

- **Restricted palette.** Slate (#0f172a) is the base colour for text, the sidebar and table headers. Emerald (#059669) marks the primary action and positive values such as net savings. Neutral greys separate surfaces. A red and an amber tone are used only for the Exceeded and Warning statuses.
- **No gradients.** Backgrounds, buttons and cards use flat fills. There are no purple or multi-colour gradients and no glowing shadows.
- **Rectangular controls.** Buttons, inputs, cards and table containers have a 6px corner radius. Pill-shaped buttons and fully rounded badges are not used.
- **SVG icons.** Icons are inline SVG with a single stroke weight and the colour of the surrounding text. The favicon is a geometric Rupee (₹) monogram in SVG. No emoji or icon fonts are loaded.
- **Status is never colour alone.** Budget statuses carry a text label (On track, Warning, Exceeded) in addition to a colour, so the meaning is available to users who cannot distinguish the colours.
- **Numbers are aligned.** Amount columns are right-aligned and use tabular figures, so digits line up in columns. Amounts are formatted with the Indian grouping through Intl.NumberFormat with the en-IN locale.
- **Plain wording.** Labels and messages state what happened (for example Budget exceeded by ₹850.00) and avoid promotional text.
- **Responsive layout.** The sidebar and the card grid collapse to a single column on narrow screens, and tables scroll horizontally inside their own container instead of widening the page.

```
:root {
  --slate-900: #0f172a;      /* text, sidebar, table headers */
  --emerald-600: #059669;    /* primary action, positive values */
  --radius: 6px;             /* every rectangular control */
}
.btn {
  background: var(--emerald-600);
  color: #fff;
  border: 1px solid var(--emerald-600);
  border-radius: var(--radius);
  padding: 8px 16px;
  font-weight: 600;
}
.btn:focus-visible { outline: 2px solid var(--slate-900); outline-offset: 2px; }
.amount { text-align: right; font-variant-numeric: tabular-nums; }
```

13.2 Interactive Chart.js Implementations

Two charts are drawn with Chart.js [10] on the dashboard. Both are re-rendered when the user changes the month, and the previous chart object is destroyed before a new one is created so that no canvas is drawn twice.

Doughnut chart: category distribution. Each segment is one expense category, and its size is the amount spent. The tooltip shows the category, the amount in INR and the share of total expenses. A legend list beside the chart repeats the category names and amounts as text, so the figures can be read without relying on the colours of the segments.

Bar chart: six-month cash flow. For each of the last six months, three bars show income, expenses and net savings. The vertical axis starts at zero, and its labels and the tooltips use the INR format. The series comes from the aggregation in Section 9.3.

```
// static/js/dashboard.js (condensed)
const inr = new Intl.NumberFormat("en-IN", {
  style: "currency", currency: "INR", minimumFractionDigits: 2
});                                          // 108000 -> ₹1,08,000.00

let trendChart;
function renderTrend(canvas, series) {
  if (trendChart) trendChart.destroy();
  trendChart = new Chart(canvas, {
    type: "bar",
    data: {
      labels: series.map(s => s.label),
      datasets: [
        { label: "Income",      data: series.map(s => s.income),
backgroundColor: "#0f172a" },
        { label: "Expenses",    data: series.map(s => s.expenses),
backgroundColor: "#94a3b8" },
        { label: "Net savings", data: series.map(s => s.net_savings),
backgroundColor: "#059669" }
      ]
    },
    options: {
      responsive: true,
      maintainAspectRatio: false,
      plugins: { tooltip: { callbacks: {
        label: ctx => `${ctx.dataset.label}: ${inr.format(ctx.parsed.y)}`
      } } },
      scales: { y: { beginAtZero: true, ticks: { callback: v => inr.format(v) } } }
    }
  });
}
```

13.3 Page Behaviour

Each page script follows the same pattern. It loads the data with fetch, writes the result into the page with textContent so that text from the server is never interpreted as HTML, and shows a message in a status region after each save, update or delete. A form is disabled while its request is pending, which

prevents a duplicate submission. An authentication error (401) from any API call sends the browser to the login page.

14. Testing and Validation

14.1 Approach

Testing combines an automated suite and a manual checklist. The automated suite in test_app.py drives the application through the Flask test client against a test database, so the development data is not changed. It covers authentication, the ledgers, budgets, goals, the server-side calculations, the advisory engine and the public pages. The cases TC-01 to TC-11 are automated, and all of them passed.

The cases TC-12 to TC-18 are manual or edge-case checks that extend the automated suite. They are derived from the implemented validation rules and routes. A result has not yet been recorded for them, so their status is stated as To Be Verified. A case is marked Passed only where a recorded result exists.

14.2 Test Cases

Table 10: Test cases and their status

ID	Feature	Expected result	Actual result	Status
TC-01	Unauthorised access	A request without a session is redirected to /login and API requests return 401	Redirected to /login as expected	Passed
TC-02	User registration validation and password hashing	A valid registration creates a user; the stored value is a hash and differs from the password; invalid input is rejected	Account created, hash stored, invalid input rejected	Passed
TC-03	User authentication and session establishment	Valid credentials start a session and give access to protected routes	Session established, protected route accessible	Passed
TC-04	Income record creation	A positive amount is stored; zero and negative amounts are rejected with 400	Valid record created, invalid amounts rejected	Passed
TC-05	Expense record logging	An expense with a valid category is stored; an unknown category is rejected	Valid record created, unknown category	Passed

ID	Feature	Expected result	Actual result	Status
			rejected	
TC-06	Budget upsert and monthly threshold detection	Saving the same month and category twice keeps one row; statuses follow the 80% and 100% thresholds	Single row retained, Warning and Exceeded statuses returned correctly	Passed
TC-07	Savings goal creation and target date tracking	A goal is created with its target amount and target date and is returned with its progress	Goal created and returned with target date	Passed
TC-08	Quick deposit to an active goal (+₹5,000)	The current amount of the goal increases by ₹5,000.00 and the progress percentage is recalculated	Current amount increased by ₹5,000.00	Passed
TC-09	Server-side net savings and savings rate	Net savings equals total income minus total expenses; savings rate equals net savings divided by total income	Values matched the expected calculation	Passed
TC-10	Gemini structured analysis and fallback response	A structurally valid analysis with the required keys and the disclaimer is returned, and the fallback result is valid when the model is not available	Valid structure returned on both paths	Passed
TC-11	Public legal pages	/privacy and /terms are served without login	Both pages rendered	Passed
TC-12	Logout	After POST /api/auth/logout, GET /dashboard redirects to /login	Not yet recorded	To Be Verified
TC-13	Login with a wrong password	The response is 401 with the message Invalid email or password. and no session is created	Not yet recorded	To Be Verified
TC-14	Registration with a duplicate email	The response is 409 and no second account is created	Not yet recorded	To Be Verified
TC-15	Expense update	After PUT and DELETE the list and the dashboard totals are	Not yet	To Be Verified

ID	Feature	Expected result	Actual result	Status
	and delete	recalculated	recorded	
TC-16	Data isolation between users	A request by user B for the expense of user A returns 404 and does not change the record	Not yet recorded	To Be Verified
TC-17	Six-month cash-flow series	The dashboard returns six consecutive months, with zero values for months without entries	Not yet recorded	To Be Verified
TC-18	Fallback with no API key	With GEMINI_API_KEY unset, POST /api/ai/analyze returns 200 with source fallback and does not call the SDK	Not yet recorded	To Be Verified

14.3 Summary of Results

Eleven of the eighteen listed cases have recorded results, and all eleven passed. These eleven cases cover the primary paths of every module. The remaining seven cases are listed so that they can be executed and recorded before the final submission. No performance or load measurements were recorded, and none are reported.

15. Results and Discussion

15.1 Functionality Demonstrated with a Realistic Indian Financial Dataset

The results in this chapter use the data loaded by seed.py for the demonstration user Ritesh Sharma (demo@financebot.local). The data represents one month of a salaried professional in an Indian city who also earns freelance and dividend income.

Income. The monthly income is ₹1,08,000.00, made up of a primary salary of ₹85,000.00, freelance income of ₹18,500.00 and a dividend of ₹4,500.00.

Expenses. The total expenses are ₹64,500.00 in the nine categories shown in Table 11. Net savings are ₹1,08,000.00 − ₹64,500.00 = ₹43,500.00, and the savings rate is 40.28%.

Budgets. Eight categories have an active limit. The Entertainment budget has a limit of ₹3,000.00 and ₹3,850.00 was spent, which is 128.33% and is flagged Exceeded. The Shopping budget has a limit of ₹6,500.00 and ₹6,400.00 was spent, which is 98.46% and is flagged Warning because it is above 80% and below 100%. The other budgets do not raise an alert.

Savings goals. The four goals are listed below.

- Emergency Reserve: ₹1,20,000.00 of ₹1,80,000.00 saved (66.67%)
- Laptop Workstation: ₹58,000.00 of ₹95,000.00 saved (61.05%)
- Cloud Certifications: ₹24,000.00 of ₹40,000.00 saved (60.00%)
- Himalayan Trek: ₹12,000.00 of ₹35,000.00 saved (34.29%)

Together the goals hold ₹2,14,000.00 of a combined target of ₹3,50,000.00, which is 61.14%. The remaining ₹1,36,000.00 is about 3.1 months of the current net savings.

15.2 Real-World Monthly Cash-Flow Demonstration

Table 11 shows the month as the dashboard presents it. The percentage of income is the quantity compared with the 50/30/20 guide, and the percentage of expenses is the category distribution drawn in the doughnut chart. Percentages are rounded to two decimals, so the nine category values of the percentage-of-income column add up to 59.71% while the exact total is 59.72%.

Table 11: Monthly cash-flow demonstration for the demo user

Line	Amount (₹)	% of income	% of expenses	Budget status
Income: Primary Salary	₹85,000.00	78.70%	–	–
Income: Freelance	₹18,500.00	17.13%	–	–
Income: Dividend	₹4,500.00	4.17%	–	–
Total income	**₹1,08,000.00**	**100.00%**	–	–
Expense: Rent	₹24,000.00	22.22%	37.21%	No alert
Expense: Food	₹13,950.00	12.92%	21.63%	No alert
Expense: Shopping	₹6,400.00	5.93%	9.92%	Warning (98.46% of ₹6,500.00)
Expense: Transport	₹5,200.00	4.81%	8.06%	No alert
Expense: Utilities	₹4,200.00	3.89%	6.51%	No alert
Expense: Entertainment	₹3,850.00	3.56%	5.97%	Exceeded (128.33% of ₹3,000.00)
Expense: Education	₹3,600.00	3.33%	5.58%	No alert
Expense: Healthcare	₹2,100.00	1.94%	3.26%	No alert
Expense: Other	₹1,200.00	1.11%	1.86%	No alert
Total expenses	**₹64,500.00**	**59.72%**	**100.00%**	1 Exceeded, 1 Warning
Net savings	**₹43,500.00**	**40.28%**	–	–

15.3 Discussion

Savings. The savings rate of 40.28% is twice the 20% guide of the 50/30/20 rule. Needs account for ₹53,050.00 (49.12% of income) and wants for ₹11,450.00 (10.60%), so both are inside their guides.

Concentration of expenses. Rent and Food together are ₹37,950.00, which is 58.84% of all expenses and 35.14% of income. Any review of the budget would therefore start with these two categories, and the doughnut chart shows this concentration directly.

Budget alerts. The two alerts are correctly separated by the thresholds. Entertainment exceeds its limit by ₹850.00, and the engine reports a Warning for Shopping, which has ₹100.00 of its limit left. The alert for Shopping is raised before the limit is crossed, which is the purpose of the 80% threshold.

Variable income. Freelance and dividend income form 21.30% of total income and may not recur in every month. The analysis of a month with only the salary of ₹85,000.00 would show expenses of 75.88% of income, which is a point that an adviser would raise.

Emergency reserve. The Emergency Reserve of ₹1,20,000.00 covers about 1.9 months of the current expenses, and the target of ₹1,80,000.00 would cover about 2.8 months. The figures can be used to judge whether the target is high enough for the user's circumstances.

Advisory output. For this data the fallback engine reports the three allocation percentages above, lists Entertainment as the overspending category with an excess of ₹850.00, recommends that Entertainment and Shopping stay within their limits, and proposes a monthly saving of ₹21,600.00, which is 20% of income. The text returned by Gemini differs in wording between runs, but it is based on the same figures and has the same structure. The user should treat both outputs as guidance.

The results show that the application converts a ledger into totals, ratios, alerts and advice without manual calculation by the user, and that the figures can be traced back to the entries that produced them.

16. Limitations

The present version is a complete working system for a single user or a small demonstration group. The following limitations apply.

- **Dependence on an external model.** The quality of the AI analysis depends on the availability of the Gemini API, on a valid API key and on the model named in the configuration. Model identifiers can be changed or withdrawn by the provider, which is why the name is read from the environment and not written in the code. The wording and the suggestions vary between runs even at a temperature of 0.2.
- **Simpler fallback.** The fallback engine applies fixed rules. It keeps the application usable but gives shorter and more general advice than the model.
- **Manual data entry.** All entries are made by the user. There is no import of bank statements, UPI history or card transactions, so the ledger is only as complete as the user's own records.
- **Fixed categories.** The nine categories cannot be changed by the user, and the assignment of categories to needs and wants in the fallback engine is a simplification. Recurring entries and split transactions are not supported.

- **Floating-point amounts.** Amounts are stored as floating-point numbers and are rounded to two decimals for display and totals. This is adequate for the values handled here but is not the preferred representation for accounting data.
- **Single currency.** Only INR is supported, and the interface does not convert between currencies.
- **Data shared with a third party.** When an analysis is requested, the aggregated monthly figures and category names are sent to the Gemini API. The user should read the Privacy Policy before using the function.
- **Security scope.** The version implements hashed passwords, session checks on every private route and user-scoped queries. Cross-site request forgery tokens, rate limiting of login attempts, email verification, two-factor authentication and enforced HTTPS are not implemented.
- **Deployment.** The application runs on the Flask development server with one SQLite file. This is suitable for demonstration and not for many simultaneous users or production use.
- **Verification.** Eleven of the eighteen listed test cases have recorded results. The remaining seven are pending, and no load or performance tests were carried out.
- **Advisory only.** The application does not consider taxes, investments, loans or insurance, and its output is not professional financial advice.

17. Future Enhancements

The enhancements below address the limitations of Chapter 16. They are ordered by area, and Table 12 gives the proposed change and the benefit expected from it.

Table 12: Proposed enhancements roadmap

Area	Proposed enhancement	Expected benefit
Data accuracy	Store amounts as integer paise or as Decimal values in the models	Exact arithmetic and totals without floating-point rounding
Data capture	Import of CSV and bank statement files, and later integration with the Account Aggregator framework	Complete ledgers with less manual entry
Flexible tracking	User-defined categories, recurring income and expenses, and split transactions	Closer match to real household finances
Analytics	Category trends over twelve months, month-to-month comparison and a forecast of month-end spending	Earlier detection of budget overruns
Alerts	Email or in-app notification when a budget passes 80% or 100%	The user is informed without opening the dashboard
Reports	Server-side export of a monthly report to PDF and CSV	Shareable and archivable reports
AI advisory	Storage of past analyses, comparison with the	Advice that takes the

Area	Proposed enhancement	Expected benefit
	previous month and a follow-up question function	history into account
Security	CSRF tokens, login rate limiting, email verification, optional two-factor authentication, HTTPS and secure cookie flags	Stronger protection of accounts and data
Deployment	A production WSGI server, a PostgreSQL database, database migrations and a container image	Support for concurrent users and repeatable installation
Quality	A continuous integration job that runs the full test suite, with the seven pending cases automated	Regressions detected before release
Localisation	Additional currencies and languages, including Hindi and Marathi	Wider use

18. Conclusion

The Personal Finance Advisor Bot meets its stated objectives. It provides an independent ledger of income and expenses in nine categories, monthly budgets with warning and exceeded thresholds, savings goals with a quick deposit action, a dashboard with Chart.js charts, and an advisory module that uses the Google Gemini API. All amounts are held and shown in Indian Rupees with Indian digit grouping.

Three design decisions shape the system. First, the application computes every figure itself in finance_service.py, and the language model only comments on figures that it is given, so the totals, ratios and alerts do not depend on the model. Second, the model reply is treated as untrusted input: it is extracted, checked against the required keys and given a fixed disclaimer. Third, a transient failure is retried with exponential backoff, and a deterministic engine based on the 50/30/20 rule produces the analysis if the model cannot be used, so the analysis endpoint always returns a valid result. In addition, passwords are stored as salted hashes and every query is restricted to the signed-in user.

The demonstration data of a user with an income of ₹1,08,000.00 and expenses of ₹64,500.00 gives net savings of ₹43,500.00 and a savings rate of 40.28%. The application correctly flags Entertainment as exceeded at 128.33% of its limit and Shopping as a warning at 98.46%, and it reports the progress of four savings goals. Eleven of the eighteen listed test cases have recorded results, and all eleven passed. The remaining seven cases and the enhancements in Chapter 17 define the work that follows.

The project shows that a conventional web application can be extended with a generative model in a controlled way, with the calculations kept in application code, the model output validated, and a fallback available. It provides a sound base for further development in the BFSI domain. The advice that it produces is informational and does not replace a qualified financial professional.

19. References

1. Python Software Foundation, "Python 3 Documentation." [Online]. Available: https://docs.python.org/3/
2. Pallets Projects, "Flask Documentation." [Online]. Available: https://flask.palletsprojects.com/
3. Pallets Projects, "Flask-SQLAlchemy Documentation." [Online]. Available: https://flask-sqlalchemy.palletsprojects.com/
4. SQLAlchemy Project, "SQLAlchemy Documentation." [Online]. Available: https://docs.sqlalchemy.org/
5. SQLite Consortium, "SQLite Documentation." [Online]. Available: https://www.sqlite.org/docs.html
6. Pallets Projects, "Werkzeug Documentation: Security Helpers." [Online]. Available: https://werkzeug.palletsprojects.com/
7. Flask-CORS Contributors, "Flask-CORS Documentation." [Online]. Available: https://flask-cors.readthedocs.io/
8. Google, "Gemini API Documentation." [Online]. Available: https://ai.google.dev/gemini-api/docs
9. Google, "Google Gen AI SDK for Python (google-genai)." [Online]. Available: https://github.com/googleapis/python-genai
10. Chart.js Contributors, "Chart.js Documentation." [Online]. Available: https://www.chartjs.org/docs/
11. E. Warren and A. W. Tyagi, *All Your Worth: The Ultimate Lifetime Money Plan*. New York, NY, USA: Free Press, 2005.
12. OWASP Foundation, "Password Storage Cheat Sheet." [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
13. R. Fielding, M. Nottingham and J. Reschke, Eds., "HTTP Semantics," RFC 9110, Internet Engineering Task Force, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110
14. T. Bray, Ed., "The JavaScript Object Notation (JSON) Data Interchange Format," RFC 8259, Internet Engineering Task Force, Dec. 2017. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8259
15. Mozilla, "Fetch API," MDN Web Docs. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
16. Pallets Projects, "Jinja Documentation." [Online]. Available: https://jinja.palletsprojects.com/
17. S. Kumar, "python-dotenv Documentation." [Online]. Available: https://saurabh-kumar.com/python-dotenv/